INF26207 – Téléinformatique

Travail de session – Les sockets

Modalité : Travail individuel ou en équipe de 2 à 3 (les mêmes équipes que pour le TP1 sauf exception)
Date de remise : vendredi le 17 avril 2026.

Spécifications techniques de la remise

Pénalité de 20% pour non-respect

- L'utilisation d'un dépôt git pour la gestion est **obligatoire** – aucun travail ne sera accepté sans dépôt montrant clairement vos *commits* = zéro sur le travail.
- Format du rapport : DOCX ← moins dix pourcent (-10%) si remis en format PDF!
  - Nom du rapport : **INF26207_TS_H2026_Equipe.docx** ← -20% sinon!
  - Interligne : 1,15 à 1,3 lignes
  - Police : Times New Roman 11 pt (ou similaire)
  - Utilisez les styles pour séparer vos sections
- Format de la remise du code (vous **devez** aussi utiliser git):
  - Le projet devra être mis dans un fichier zip, 7z, tar.gz ou rar : **INF26207_TS_H2026_Equipe.extension** ← -20% si non-conforme!
  - Utilisez une structure de dossier et des noms de fichiers appropriés pour votre projet/code

Objectifs

- Utilisation des sockets UDP en Python 3.10+
- Implémenter un service de transfert de fichier fiable
- Se familiariser avec la notion de sérialisation binaire
- Concevoir un en-tête applicatif
- Gérer les:
  - pertes; corruption; doublons;
  - retransmissions; reprise de transfert.

1 Énoncé

Vous devez implémenter une application console de type **serveur de sauvegarde** comportant un client et un serveur capables d'effectuer un transfert fiable de fichier à l'aide du protocole UDP, malgré la présence de pertes et de corruption de segments. Une classe `usocket` vous est fournie à cette fin et son utilisation est obligatoire.

VOUS NE DEVEZ PAS MODIFIER usocket.pyc.

Comme nous l'avons vu, contrairement à un protocole comme TCP, UDP ne fournit pas nativement :

- la retransmission;
- les acquittements;
- la détection de doublons;
- la reprise de transfert interrompu.

Le but du travail est donc de vous concentrer sur la logique de fiabilité du protocole et non sur la simulation du réseau lui-même.

- Vous devez implémenter votre solution à l'aide des librairies de base.
- N'utilisez pas plein de librairies externes, à la rigueur, se limiter à `tqdm` pour les barres de progrès.
- Si vous utilisez d'autres librairies, vous assurer de fournir un fichier requirements.txt.

Votre solution doit fonctionner **en mode console uniquement** – je ne veux pas gérer de code GUI!

Usocket (socket non-fiable)

Le fichier `usocket.pyc` (en fait, trois versions selon votre version de Python) vous est fourni et son utilisation est **obligatoire** pour les communications réseau du travail. Par exemple, si vous utilisez Python 3.11, téléchargez `usocket31.pyc` et renommez le `usocket.pyc`, ensuite :

```
import usocket
```

Vous ne devez pas :

- modifier usocket.pyc;
- remplacer usocket.pyc par votre propre implémentation;
- utiliser directement socket.socket() à la place de usocket.py pour le protocole demandé.

La classe fournie simule notamment :

- des **pertes** de segments;
- des **corruptions** aléatoires de segments.

2 Fonctionnement général

Vous devez implémenter un serveur de sauvegarde simple.

Les commandes reconnues sont les suivantes :

- **`open adresse_ip`** : initie une connexion au serveur spécifié par l'adresse IP
- **`ls`** : retourne la liste des fichiers disponibles sur le serveur
- **`put nom_de_fichier`** : téléverse le fichier spécifié vers le serveur
- **`bye`** : termine la connexion au serveur

Le serveur écoute sur le port 4242 pour une demande de connexion de la part d'un client.

Lorsqu'un client le contacte, un processus de poignée de main devrait être initié pour confirmer la connexion (pensez au Three-Way Handshake du protocole TCP). Cette négociation doit notamment permettre de s'entendre sur la taille des morceaux et le nombre maximal de morceaux avant acquittement.

Durant le processus de la poignée de main, le client et le serveur devront s'entendre sur :

- la taille maximale de morceaux de fichiers envoyés/reçus;
- le nombre maximal de morceaux pouvant être envoyés avant attente d'un accusé de réception.

Une fois la connexion établie avec le client, le serveur attend une commande. Si un transfert de fichier est initié, le transfert sera effectué en découpant le fichier en morceaux et en utilisant plusieurs envois.

Une fois le fichier transmis, le serveur attend la prochaine commande.

3 Structure attendue du projet

Votre archive doit respecter minimalement une structure semblable à celle-ci :

```
projet/
├── usocket.pyc         # Fourni – OBLIGATOIRE
├── usocket.pyi         # Fourni – pour aider Intellisense
├── client.py           # Console
├── serveur.py          # Port 4242, console
├── config.ini          # Paramètres réseau et connexion
├── requirements.txt    # Si nécessaire
├── README.md           # Doc. Lancement + tests + vérification d'intégrité
├── tests/              # ≥3 binaires ≥200 Ki< à tester de votre côté
└── sauvegardes/        # Dossier de sauvegarde des fichiers
```

3.1 Contraintes associées

- serveur.py doit écouter sur le **port 4242**.
- Les fichiers reçus par le serveur doivent être enregistrés dans un dossier ./sauvegardes/.

- Votre README.md doit expliquer :
    - comment lancer le serveur;
    - comment lancer le client;
    - comment exécuter un transfert;
    - comment tester resume;
    - comment vérifier l'intégrité finale du fichier.
- config.ini doit être utilisé pour au moins la fiabilité du réseau, le taux de corruption, le timeout et les paramètres de connexion.

4 Détails techniques

Le client contacte le serveur afin de recevoir le fichier. Le fichier lui parvient en plusieurs blocs (segments). Le client sera averti d'une façon quelconque de la fin du fichier.

Vous devez donc établir une syntaxe et structure de messages cohérente que le client et le serveur peuvent envoyer/recevoir.

Inspirez-vous des protocoles TCP, HTTP pour la conception et la définition de la structure de vos messages, par exemple :

```
+---+--------+--------+-+--------+---------+
|CMD|<PARAM1>|<PARAM2>|…|<PARAMN>|<DONNEES>|
+---+--------+--------+-+--------+---------+
```

En somme, vous devez concevoir vos propres en-tête application (encapsulation, PCI [Protocol Control Information] ← module 2).

4.1 Encodage

!!! N'utilisez pas l'encodage textuel (ASCII, UTF, ANSI) pour transmettre votre fichier ni pour vos en-têtes !!!

Vos en-têtes doivent être encodés en **binaire** avec `struct`.

Exemple de format pour l'en-tête :

```
HEADER_FMT = "!BBIIHI"
```

Exemple de correspondance selon le format proposé:

- ver : version du protocole (uint8)
- type : type de message (uint8)
- seq : numérde séquence (uint32)
- ack : numérd'acquittement (uint32)
- payload_len : longueur utile (uint16)
- checksum32 : checksum (uint32)

La signification des valeurs vous appartient, mais elle devra être documentée dans le rapport.

4.1.1 À propos de struct

Tous les en-têtes doivent être encodés en **ordre réseau**. En Python, cela se fait avec le préfixe `!`, par exemple :

```
struct.pack("!I", x)
```

Sans ce préfixe, `struct` peut utiliser l'endianness et l'alignement natifs de la machine, ce qui est interdit pour un protocole réseau.

4.2 Taille des blocs

Les blocs transmis ne doivent pas contenir plus d'octets du fichier que ce qui a été convenu lors de l'établissement de la connexion.

Le fichier transféré doit faire **au moins 200 Kio**.

4.3 Intégrité

Votre application devra vérifier que le fichier reçu est identique à celui envoyé.

Vous devez donc prévoir :

- un checksum par segment; ← si vous vérifiez juste à la fin, votre transfert va échouer
- une vérification finale de l'intégrité du fichier complet.

Un segment corrompu **ne doit pas être accepté** comme valide.

4.4 Réseau non fiable

Votre réseau n'est pas fiable. Dans ce TP, cette faiblesse est simulée par la classe `usocket`. Le wrapper fourni simule des pertes d'envoi selon une fiabilité configurable et une corruption des segments selon un taux de corruption configurable, ce qui est précisément conçu pour tester retransmissions, timeouts et checksums.

Vous devez utiliser `usocket.pyc`[1] pour tous les envois/réceptions liés au protocole demandé.

4.5 Fichier de configuration

Vous devez utiliser un fichier `config.ini`, lu à l'aide du module standard `configparser`.

Import attendu :

```
import configparser
```

Exemple :

```
[RESEAU]
fiabilite=0.95
taux_corruption=0.02
timeout=3.0 ; 1-3 secondes pour débogage initial, mais 0.1 recommandé sinon
max_reprises=5
```

[1] Oui, je me répète, mais c'est pour que les attentes soient claires

```
[CONNEXION]
client_mss_propose=1024
serveur_mss_propose=1024
n_propose=4
```

- `fiabilite` : paramètre transmis à la socket non fiable fournie;
- `timeout` : délai d'attente maximal avant retransmission;
- `max reprises` : nombre maximal de tentatives consécutives avant abandon;
- `[x]_mss_propose` : taille maximale proposée pour les données utiles d'un segment;
- `n_propose` : taille de fenêtre proposée, soit le nombre maximal de segments envoyés avant attente d'un ACK.

4.6 Acquittements

La réception de chaque bloc de N morceaux d'information (segments) doit être confirmée par un accusé de réception.

À défaut de recevoir l'accusé de réception d'un bloc de N morceaux dans les 3 secondes suivantes (valeur définie dans votre configuration), les morceaux seront réexpédiés par le client (une fois le débogage initial terminé, je vous suggère fortement de configurer le timeout à ~10ms max, sinon le transfert va prendre énormément de temps)

Si après 5 tentatives d'envoi consécutives on échoue toujours, on termine le transfert et on avertit l'utilisateur de l'échec.

Il n'y a pas de réexpédition spéciale d'un accusé de réception. Toutefois, si un de ceux-ci se perd, le bloc de N morceaux original sera réexpédié après le délai prévu, et un nouvel accusé de réception sera donc émis à sa réception.

4.7 Doublons

La réexpédition et la perte de messages peuvent occasionner des doublons.

Pour reconnaître le dédoublement des morceaux, il faudra les numéroter d'une façon quelconque. Lorsqu'on reçoit un doublon, on peut alors le détecter et l'ignorer.

4.8 Reprise [optionnel BONUS + 10 %]

Vous devez supporter une commande additionnelle de reprise de téléversement :

```
resume nom_de_fichier
```

L'idée est qu'un transfert interrompu puisse reprendre à partir du dernier segment ou du dernier point valide connu du serveur, sans recommencer tout le fichier.

Vous devrez donc prévoir un mécanisme cohérent permettant :

- au client de demander où en est le transfert;
- au serveur d'indiquer ce qu'il considère valide;
- au client de reprendre à partir de ce point.

4.9 Fin de transfert

Vous aurez aussi à trouver un moyen d'informer le serveur puis le client que tout le fichier a été transmis correctement.

4.10 Tests

Vous pouvez utiliser une adresse IPv4 de rebouclage (`127.0.0.x`) pour implémenter et tester vos applications.

5 Conseil

Commencez simple :

1. faire open et bye
2. établir votre en-tête
3. envoyer un seul segment
4. ajouter/vérifier le checksum
5. ajouter les ACK
6. ajouter le fenêtrage
7. ajouter resume (optionnel - bonus)

6 Livrables

Vous devez remettre un rapport (maximum de 4 pages + annexes) décrivant le fonctionnement de vos applications, vos choix de conception et leurs justifications.

Le rapport devrait inclure :

- la structure et la syntaxe de vos messages;
- une **table PCI** indiquant les champs, leur taille et leur rôle;
- la signification des types de messages;
- la justification du checksum choisi;
- la justification des valeurs par défaut pour MSS et N;
- l'explication du rôle de seq dans votre protocole;
- un court résumé du mécanisme de reprise.

Lors de la correction, si un doute est soulevé quant à l'authenticité de votre travail, vous aurez à faire fonctionner votre application, répondre à des questions sur sa conception et, peut-être, apporter quelques modifications à votre code.

6.1 Barème

- **Fonctionnements corrects du client et du serveur ➔** 40 %
  - Ne plante pas
  - Résultat attendu

    - Respecte les requis
    - put fonctionne
    - Gère les pertes, corruptions et doublons
- **Qualité du rapport ➔** 10 %
    - Clarté et organisation (5 %)
        - Introduction et conclusion claires (2 %)
        - Les sections sont bien organisées (3 %)
    - Explication des choix de conception (3 %)
        - Justification des choix techniques (2 %)
        - Explication des compromis (1 %)
    - Description des difficultés rencontrées et solutions (2 %)
        - Présentation des problèmes rencontrés (1 %)
        - Solutions appliquées pour les résoudre (1 %)
- **Choix de conception ➔** 25 %
    - Détails d'implémentation : entêtes, structures, etc.
    - Logique
    - Cohérence du protocole
    - Gestion de la reprise
- **Qualité de la programmation ➔** 15 %
    - Commentaires
    - Propreté
    - Conventions
    - Structure des fichiers, des dossiers
    - Etc.
- **Qualité des interfaces, respect des consignes, etc. ➔** 10 %

Notes :

```
# Pas besoin de faire des unit test, à moins que ça vous tente, mais je
ne les vérifierai pas
# L'application demandée est simple, un programme trop complexe
pourrait être pénalisé
# Commentez votre code, en particulier si vous définissez des
constantes ou autres, je n'aime pas trop jouer aux devinettes. Trop de
commentaires sont mieux que pas assez de commentaires dans ce cas.
```